8:3 Priority Encoder

y_7	y_6	y_5	y_4	y_3	y_2	y_1	y_0	a	b	c	d
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	X	0	0	1	1
0	0	0	0	0	1	X	X	0	1	0	1
0	0	0	0	1	X	X	X	0	1	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	1	X	X	X	X	X	1	0	1	1
0	1	X	X	X	X	X	X	1	1	0	1
1	X	X	X	X	X	X	X	1	1	1	1

- a = y7 | (y6 & ~y7) | (y5 & ~y6 & ~y7) | (y4 & ~y5 & ~y6 & ~y7)
- b = y7 | (y6 & ~y7) | (y3 & ~y4 & ~y5 & ~y6 & ~y7) | (y2 & ~y3 & ~y4 & ~y5 & ~y6 & ~y7)
- c = y7 | (y5 & ~y6 & ~y7) | (y3 & ~y4 & ~y5 & ~y6 & ~y7) | (y1 & ~y2 & ~y3 & ~y4 & ~y5 & ~y6 & ~y7)
- valid = d = y0 | y1 | y2 | y3 | y4 | y5 | y6 | y7;

8:3 Priority Encoder using Dataflow

```
module priority_encoder_8to3_df (

input  [7:0] in,

output [2:0] out,

output valid);

assign out[2] = in[7] | (in[6] & ~in[7]) | (in[5] & ~in[6] & ~in[7]) | (in[4] & ~in[5]
& ~in[6] & ~in[7]);

assign out[1] = in[7] | (in[6] & ~in[7]) | (in[3] & ~in[4] & ~in[5] & ~in[6] &
~in[7]) | (in[2] & ~in[3] & ~in[4] & ~in[5] & ~in[6] & ~in[7]);

assign out[0] = in[7] | (in[5] & ~in[6] & ~in[7]) | (in[3] & ~in[4] & ~in[5] &
~in[6] & ~in[7]) | (in[1] & ~in[2] & ~in[3] & ~in[4] & ~in[5] & ~in[6] & ~in[7]);

assign valid =in[0]|in[1]|in[2]|in[3]|in[4]|in[5]|in[6]|in[7];

endmodule
```

8:3 Priority Encoder using Dataflow

```
module priority_encoder_8to3_df (
input  [7:0] in,
output [2:0] out,
output valid);
assign out = (in[7]) ? 3'b111 :
             (in[6]) ? 3'b110 :
             (in[5]) ? 3'b101 :
             (in[4]) ? 3'b100 :
             (in[3]) ? 3'b011 :
             (in[2]) ? 3'b010 :
             (in[1]) ? 3'b001 :
             (in[0]) ? 3'b000 :
                   3'b000;
assign valid =in[0]|in[1]|in[2]|in[3]|in[4]|in[5]|in[6]|in[7];
endmodule
```

8:3 Priority Encoder Test Bench

```
module priority_encoder_8to3_df_tb;
reg [7:0] in;
wire [2:0] out;
wire       valid;
priority_encoder_8to3_df p (.in(in),  .out(out), .valid(valid));
initial begin
in = 8'b00000000; #100;
in = 8'b00000001; #100;
in = 8'b00000010; #100;
in = 8'b00000100; #100;
in = 8'b00001000; #100;
in = 8'b00010000; #100;
in = 8'b00100000; #100;
in = 8'b01000000; #100;
in = 8'b10000000; #100;
in = 8'b11111111; #100;
in = 8'b00010010; #100;
$finish;
end
endmodule
```

Behavioral modeling

- Behavioral modeling represents digital circuits at a functional and algorithmic level.
- It is used mostly to describe sequential circuits, but can be used also to describe combinational circuits.
- Behavioral descriptions use the keyword always followed by a list of procedural assignment statements.
- The target output of procedural assignment statements must be of the reg data type. Contrary to the wire data type, where the target output of an assignment may be continuously updated.
- A reg data type retains its value until a new value is assigned.

Behavioral modeling

- Uses procedural blocks and procedural statements
- There are two types of procedural blocks in Verilog

  1. **The initial block**

     ✧ Executes the enclosed statement(s) one time only

  2. **The always block**

     ✧ Executes the enclosed statement(s) repeatedly until simulation terminates

- The body of the initial and always blocks is procedural
  - Can enclose one or more procedural statements
  - Procedural statements are surrounded by begin ... End
- Multiple procedural blocks can appear in any order inside a module and run in parallel inside the simulator

initial Statement

- Once executable only
- Execution starts at 0 simulation time
- Execution stops when the last statement is executed
- Parallel execution in case of multiple initial blocks
- Multiple statements inside an initial block need to be grouped with **begin and end statements**

Example:

```
module stimulus;
reg x, y, a, b, m;
initial
m = 1'b0;  // single statement; does not need to be grouped
Initial
begin
#5  a = 1'b1;  // multiple statements; need to be grouped
#25 b = 1’b0;
end
Initial
begin
#10 x = 1'b0;
#25 y = 1’b1;
end
initial
#50 $finish;
endmodule
```

Simulation result

time	statement executed
0	m = 1'b0;
5	a = 1'b1;
10	x = 1'b0;
30	b = 1'b0;
35	y = 1'b1;
50	$finish;

Example: cont.

- The three initial statements start to execute in parallel at time 0.
- If a delay #<delay> is seen before a statement, the statement is executed <delay> time units after the current simulation time.

always Statement

- Repeats continuously throughout the duration of simulation time
- Like initial block it is also concurrent in nature and starts at 0 simulation time
- Parallel execution in case of multiple always blocks
- A deadlock condition will be created if an always construct has no control for simulation time

always Construct

- The statements which come under the always construct constitute the always block.
- The always block starts at time 0, and keeps on executing all the simulation time.
- It works like an infinite loop.
- always blocks execute over and over again; in other words, as the name suggests, it executes always.

Structured Procedures: always statement

- Starts at time 0
- Execute the statements in a looping fashion
- Multiple always blocks, execute in parallel

  All start at time 0

Syntax:

```
always @(sensitivity list)
begin
    // behavioral statements
end
```

"always" block example:

```
module and_or_gate (out, in1, in2, in3);

input   in1, in2, in3;

output  out;

reg     out;

always @(in1 or in2 or in3)

begin

out = (in1 & in2) | in3;

end

endmodule
```

BASIC GATES (ALL GATES IN ONE PROGRAM)

```
module gates(a,b,c,d,e,f,g,h,i,j);
input a,b;
output c,d,e,f,g,h,i,j;
assign  c=a&b;
assign  d=a|b;
assign  e=~a;
assign  f=a;
assign  g=~(a|b);
assign  h=a^b;
assign  i=~(a^b);
assign  j=~(a&b);
endmodule
```

```
module gates(a,b,c,d,e,f,g,h,i,j);
input a,b;
output c,d,e,f,g,h,i,j;
reg c,d,e,f,g,h,i,j;
always @(a,b)
begin
c=a&b;
d=a|b;
e=~a;
f=a;
g=~(a|b);
h=a^b;
i=~(a^b);
j=~(a&b);
end
endmodule
```

```
module gates(a,b,c,d,e,f,g,h,i,j);
input a,b;
output c,d,e,f,g,h,i,j;
and (c,a,b);
or(d,a,b);
not(e,a);
buf(f,a);
nor(g,a,b);
xor(h,a,b);
xnor(i,a,b);
nand(j,a,b);
endmodule
```

Conditional statements

- if-else constructs
  - like C, except that instead of open and close brackets { ... } use keywords begin ... end to group multiple assignments associated with a given condition
  - begin ... end are not needed for single assignments
- case constructs
  - similar to C switch statements, selecting one of multiple options based on values of a single selection signal
- for (i = 0; i < 10; i = i + 1) statements
- repeat (count) statements // repeat statements “count” times
- while (abc) statements // repeat statements while abc “true” (non-0)

case statement

- The if-else statement provides the means for choosing an alternative based on the value of an expression.
- When there are many possible alternatives, the code based on this statement may become awkward to read.
- Instead, it is often possible to use the Verilog case statement which is defined as

```
case (expression)
alternative1: statement;
alternative2: statement;   .   .   .
alternativej: statement;
[default: statement;]
endcase
```

Behavioral Modeling – Half Adder

```
//Half Adder : Behavioral description

module ha_beh (x,y,s,c);
  input x,y;
  output s,c;
  reg s, c;
  always @(x or y)
    begin
      s = x ^ y;
      c = x & y;
    end
endmodule
```

Behavioral Modeling – Half Adder – using case

```
module half_adder(A, B, S, Cout);
input A,B;
output S, Cout;
reg S, Cout;
 always @(A or B)
  begin
   case (A | B)
     2'b00: begin S = 0; Cout = 0; end
     2'b01: begin S = 1; Cout = 0; end
     2'b10: begin S = 1; Cout = 0; end
     2'b11: begin S = 0; Cout = 1; end
   endcase
  end
endmodule
```

Behavioral Modeling – Half Subtractor

```
//Half Subtractor : Behavioral description

module half_sub_beh(diff,borr,a,b);
input a,b;
output diff,borr;
reg diff,borr;
always@(a or b)
begin
diff=a^b;
borr=~a&b;
end
endmodule
```

Behavioral Modeling – Full Adder

```
module full_add_beh(sum,carry,a,b,c);
input a,b,c;
output sum,carry;
reg sum,carry;
wire w1,w2,w3;
always@(a,b,c)
begin
w1=a^b;
sum=w1^c;
w2=a&b;
w3=w1&c;
carry=w2|w3;
end
endmodule
```

Behavioral Modeling 2:1 Mux

- The procedural assignment statements inside the **always** block are executed every time there is a change in any of the variable listed after the @ symbol. (Note that there is no ";" at the end of **always** statement)

```
//Behavioral description of 2-to-1-line multiplexer
module mux2x1_bh(A,B,select,OUT);
  input A,B,select;
  output OUT;
  reg OUT;
  always @(select or A or B)
  begin
      if (select == 0) OUT = A;
      else OUT = B;
  end
endmodule
```

2:1 Mux using conditional operator

```
module mux2to1 (w0, w1, s, f);

input  w0, w1, s;

output reg f;

always @(w0, w1, s)

f = s ? w1 : w0;

endmodule
```

Behavioral Modeling 4-to-1 multiplexer

```
//Behavioral description of 4-to-1 line mux
module mux4x1_bh (i0,i1,i2,i3,select,y);
  input i0,i1,i2,i3;
  input [1:0] select;
  output y;
  reg y;
  always @(i0 or i1 or i2 or i3 or select)
        case (select)
          2'b00: y = i0;
          2'b01: y = i1;
          2'b10: y = i2;
          2'b11: y = i3;
        endcase
endmodule
```

Behavioral Modeling 4-to-1 multiplexer

- In 4-to-1 line multiplexer, the select input is defined as a 2-bit vector and output y is declared as a reg data.
- The always block has a sequential block enclosed between the keywords **case** and **endcase**.
- The block is executed whenever any of the inputs listed after the @ symbol changes in value.

```
// function modeled by its "behavior"
module  MUX2 (A,B,C,D,S,Z1,Z2);
input A,B,C,D;                          //mux inputs
input [1:0] S;                          //mux select inputs
output Z;                               //mux output
always   //evaluate block whenever there are changes in S,A,B,C,D
      begin   //if-else form
            if      (S == 2'b00) Z1 = A;        //select input A for S=00
            else if (S == 2'b01) Z1 = B;        //select input B for S=01
            else if (S == 2'b10) Z1= C;         //select input C for S=10
            else if (S == 2'b11) Z1 = D;        //select input D for S=11
            else                 Z1 = x;        //otherwise unknown output
      end
//assign statement using the conditional operator (in lieu of always block)
assign Z2 = (S == 2'b00) ? A:                   //select A for S=00
            (S == 2'b01) ? B:                   //select B for S=01
            (S == 2'b10) ? C:                   //select C for S=10
            (S == 2'b11) ? D:                   //select D for S=11
                           x;                   //otherwise default to x
endmodule
```

```
//equivalent case statement form
case (S)
    2'b00:  Z1 = A;
    2'b01:  Z1 = B;
    2'b10:  Z1 = C;
    2'b11:  Z1 = D;
    default: Z1 = x;
endcase
```

Synthesis may insert latches when defaults not specified.

Behavioral Modeling – 4:1 Mux

```
module mux4to1 (w0, w1, w2, w3, S, f);
    input w0, w1, w2, w3;
    input [1:0] S;
    output reg f;

    always @(*)
        if (S == 2'b00)
            f = w0;
        else if (S == 2'b01)
            f = w1;
        else if (S == 2'b10)
            f = w2;
        else
            f = w3;

endmodule
```

```
module mux4to1 (W, S, f);
    input [0:3] W;
    input [1:0] S;
    output reg f;

    always @(W, S)
        if (S == 0)
            f = W[0];
        else if (S == 1)
            f = W[1];
        else if (S == 2)
            f = W[2];
        else
            f = W[3];

endmodule
```

Behavioral Modeling – 4:1 Mux

```
module mux4to1 (W, S, f);
    input [0:3] W;
    input [1:0] S;
    output reg f;

    always @(W, S)
        case (S)
            0: f = W[0];
            1: f = W[1];
            2: f = W[2];
            3: f = W[3];
        endcase

endmodule
```

```
module MUX4 (A,B,C,D,S0,S1,Z);
input A,B,C,D,S0,S1;
output Z;
wire Z1,Z2;
always
    begin
            if      ((S1 == 1'b0) && (S0 == 1'b0)) Z = A;
            else if ((S1 == 1'b0) && (S0 == 1'b1)) Z = B;
            else if ((S1 == 1'b1) && (S0 == 1'b0)) Z = C;
            else                                   Z = D;
    end
endmodule
```

1:4 Demux Verilog Code

```
module demux_1_4(
  input [1:0] sel,
  input  i,
  output reg y0,y1,y2,y3);

  always @(*)
begin
    case(sel)
      2'h0: {y0,y1,y2,y3} = {i,3'b0};
      2'h1: {y0,y1,y2,y3} = {1'b0,i,2'b0};
      2'h2: {y0,y1,y2,y3} = {2'b0,i,1'b0};
      2'h3: {y0,y1,y2,y3} = {3'b0,i};
      default: $display("Invalid sel input");
    endcase
  end
endmodule
```

Testbench Code

```
module tb;
  reg [1:0] sel;
  reg i;
  wire y0,y1,y2,y3;
  demux_1_4 demux(sel, i, y0, y1, y2, y3);
  initial begin
    $monitor("sel = %b, i = %b -> y0 = %0b, y1 = %0b ,y2 = %0b, y3 = %0b", sel,i,
y0,y1,y2,y3);
    sel=2'b00; i=0; #1;
    sel=2'b00; i=1; #1;
    sel=2'b01; i=0; #1;
    sel=2'b01; i=1; #1;
    sel=2'b10; i=0; #1;
    sel=2'b10; i=1; #1;
    sel=2'b11; i=0; #1;
    sel=2'b11; i=1; #1;
  end
endmodule
```

Behavioral Modeling – 2:4 Decoder

```
module dec2to4 (W, En, Y);
    input [1:0] W;
    input En;
    output reg [0:3] Y;

    always @(W, En)
        case ({En, W})
            3'b100: Y = 4'b1000;
            3'b101: Y = 4'b0100;
            3'b110: Y = 4'b0010;
            3'b111: Y = 4'b0001;
            default: Y = 4'b0000;
        endcase

endmodule
```

```
module dec2to4 (W, En, Y);
    input [1:0] W;
    input En;
    output reg [0:3] Y;

    always @(W, En)
    begin
        if (En == 0)
            Y = 4'b0000;
        else
            case (W)
                0: Y = 4'b1000;
                1: Y = 4'b0100;
                2: Y = 4'b0010;
                3: Y = 4'b0001;
            endcase
    end

endmodule
```

Behavioral Modeling – 3:8 Decoder

```
module binary_decoder(D, y);
  input [2:0] D;
  output reg [7:0] y;
    always@(D) begin
    y = 0;
    case(D)
      3'b000: y[0] = 1'b1;
      3'b001: y[1] = 1'b1;
      3'b010: y[2] = 1'b1;
      3'b011: y[3] = 1'b1;
      3'b100: y[4] = 1'b1;
      3'b101: y[5] = 1'b1;
      3'b110: y[6] = 1'b1;
      3'b111: y[7] = 1'b1;
      default: y = 0;
    endcase
  end
endmodule
```

Test-Bench 3:8 Decoder

```
module tb;
  reg [2:0] D;
  wire [7:0] y;
  binary_decoder  bin_dec(D, y);
  initial
begin
    $monitor("D = %b -> y = %0b", D, y);
    repeat(5) begin
      D=$random;
      #100;
    end
  end
endmodule
```

8:3 Priority Encoder using Behavioral

```
module priority_encoder_8to3 (
input  [7:0] in,
output reg [2:0] out,
output reg  valid);
always @(*)
begin
valid = 1'b1;
casex (in)
8'b1xxxxxxx: out = 3'b111;
8'b01xxxxxx: out = 3'b110;
8'b001xxxxx: out = 3'b101;
8'b0001xxxx: out = 3'b100;
8'b00001xxx: out = 3'b011;
8'b000001xx: out = 3'b010;
8'b0000001x: out = 3'b001;
8'b00000001: out = 3'b000;
default: begin
out   = 3'b000;
valid = 1'b0;
 end
endcase
end
endmodule
```

Hierarchical code – 4:16 Decoder

```
module dec4to16 (W, En, Y);
    input [3:0] W;
    input En;
    output [0:15] Y;
    wire [0:3] M;

    dec2to4 Dec1 (W[3:2], M[0:3], En);
    dec2to4 Dec2 (W[1:0], Y[0:3], M[0]);
    dec2to4 Dec3 (W[1:0], Y[4:7], M[1]);
    dec2to4 Dec4 (W[1:0], Y[8:11], M[2]);
    dec2to4 Dec5 (W[1:0], Y[12:15], M[3]);

endmodule
```

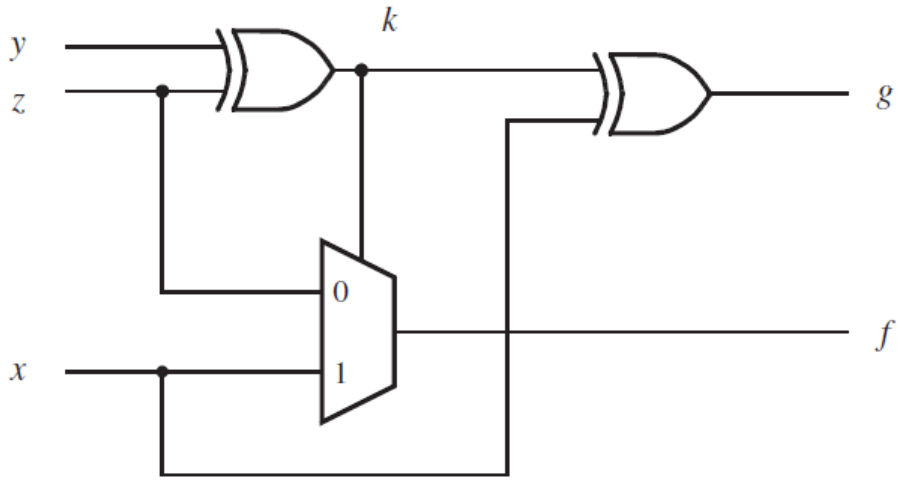
Hierarchical code – 16:1 Mux

```
module  mux16to1 (W, S, f);
    input  [0:15] W;
    input  [3:0] S;
    output  f;
    wire  [0:3] M;

    mux4to1  Mux1  (W[0:3], S[1:0], M[0]);
    mux4to1  Mux2  (W[4:7], S[1:0], M[1]);
    mux4to1  Mux3  (W[8:11], S[1:0], M[2]);
    mux4to1  Mux4  (W[12:15], S[1:0], M[3]);
    mux4to1  Mux5  (M[0:3], S[3:2], f);

endmodule
```

Example




```
module  f_g (x, y, z, f, g);
    input x, y, z;
    output f, g;
    wire k;

    assign k = y ^ z;
    assign g = k ^ x;
    assign f = (~k & z) | (k & x);

endmodule
```

Consider the circuit shown in Figure 2.72. It includes two copies of the 2-to-1 multiplexer circuit from Figure 2.33, and the adder circuit from Figure 2.12. If the multiplexers' select input $m = 0$, then this circuit produces the sum $S = a + b$. But if $m = 1$, then the circuit produces $S = c + d$. By using multiplexers, we are able to *share* one adder for generating the two different sums $a + b$ and $c + d$. Sharing a subcircuit for multiple purposes is commonly-used in practice, although usually with larger subcircuits than our one-bit adder. Write Verilog code for the circuit in Figure 2.72. Use the hierarchical style of Verilog code illustrated in Figure 2.47, including two instances of a Verilog module for the 2-to-1 multiplexer subcircuit, and one instance of the adder subcircuit.

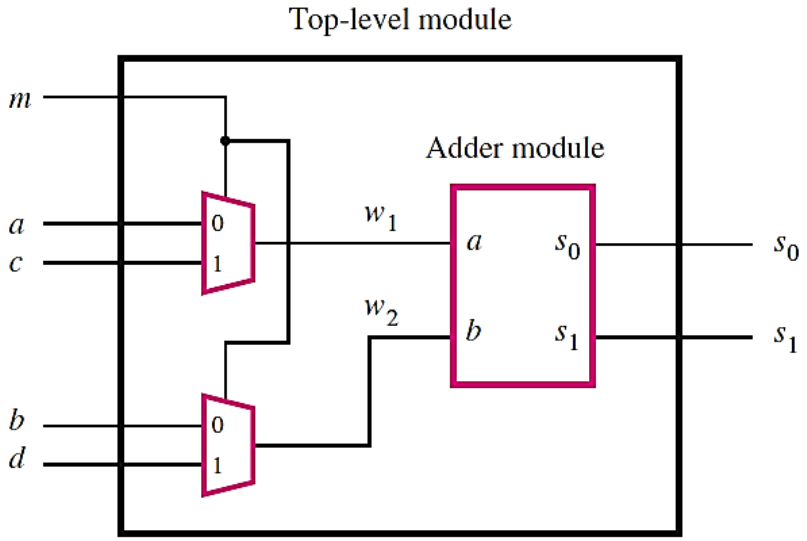


Figure 2.72 The circuit for Example 2.30.

Example

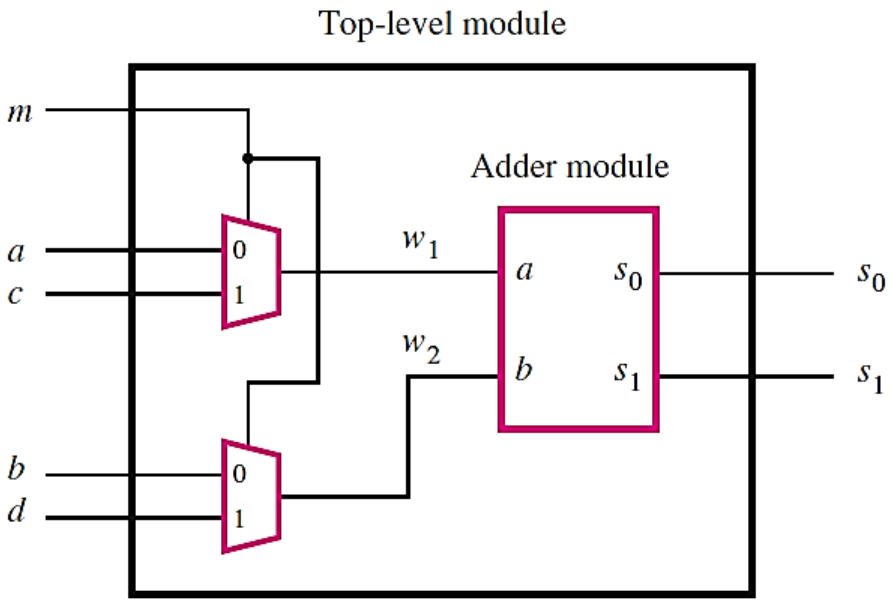


Figure 2.72 The circuit for Example 2.30.

```
module shared (a, b, c, d, m, s1, s0);
    input a, b, c, d, m;
    output s1, s0;
    wire w1, w2;
    mux2to1 U1 (a, c, m, w1);
    mux2to1 U2 (b, d, m, w2);
    adder U3 (w1, w2, s1, s0);
endmodule

module mux2to1 (x1, x2, s, f);
    input x1, x2, s;
    output f;
    assign f = (~s & x1) | (s & x2);
endmodule
module adder (a, b, s1, s0);
    input a, b;
    output s1, s0;
    assign s1 = a & b;
    assign s0 = a ^ b;
endmodule
```